



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-2

8-Queen Problem

Aim

To write a Python program to solve the 8-Queen problem using the Backtracking algorithm.

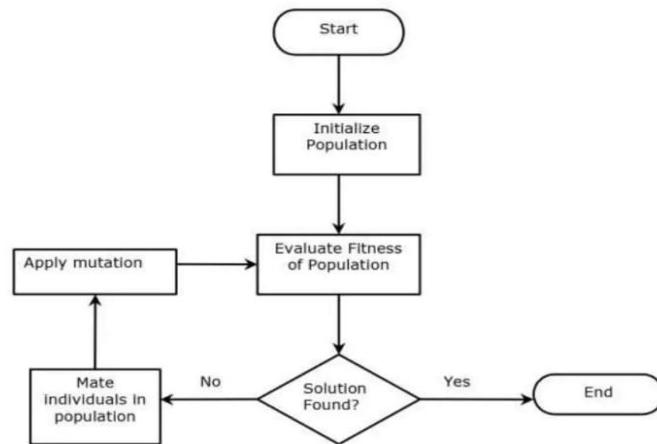
Objective

To place 8 queens on an 8×8 chessboard such that no two queens attack each other.

Algorithm

1. Start.
2. Create an 8×8 board initialized with 0.
3. Begin placing queens column by column.
4. For each row in the current column, check whether placing a queen is safe:
 - a. No queen in the same row.
 - b. No queen in the upper-left diagonal.
 - c. No queen in the lower-left diagonal.
5. If safe, place the queen and move to the next column.
6. If all 8 queens are placed, print the solution.
7. If no row is safe, remove the previously placed queen (backtrack) and try the next row.
8. Stop when a solution is found or all possibilities are exhausted.

Flowchart



Code

```
N = 8
board = [-1] * N
def safe(r, c):
    for i in range(c):
        if board[i] == r or abs(board[i] - r) == abs(i - c):
            return False
    return True
def solve(c):
    if c == N:
        for r in range(N):
            print(" ".join("Q" if board[x] == r else "." for x in range(N)))
        return True
    for r in range(N):
        if safe(r, c):
```

```

        board[c] = r
    if solve(c):
        return True

return False
solve(0)

```

Output

The screenshot shows a Python IDE with two windows. The left window displays the source code for solving the 8-Queen problem using backtracking. The right window shows the execution output, which includes the restart path and the final solution board.

```

CSA1712 exp 2.py - C:/Users/sowmi/OneDrive/Documents/New folder (2)/CSA1712 exp 2.py (3.14.2)
File Edit Format Run Options Window Help
N = 8
def print_board(board):
    for row in board:
        print(" ".join("Q" if x == 1 else "." for x in row))

def is_safe(board, row, col):
    # Check row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check lower diagonal on left side
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

def solve_queen(board, col):
    # If all queens are placed
    if col >= N:
        return True

    # Try each row in current column
    for row in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve_queen(board, col + 1):
                return True

    # Backtrack
    board[row][col] = 0

IDE Shell 3.14.2
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/sowmi/OneDrive/Documents/New folder (2)/CSA1712 exp 2.py ==
Solution for 8-Queen Problem:
Q . . . . .
. . . . . Q
. . . Q . .
. . . . . Q
. Q . . . .
. . . Q . .
. . . . . Q
. . Q . . .
>>>

```

Result

The 8-Queen problem is solved successfully using the Backtracking algorithm.

Conclusion

The Backtracking algorithm places all 8 queens on the chessboard without any conflicts.